

Clase 6

Intro análisis Solidity

Modelo de amenazas y herramientas

Tesis de la clase

El modelo de cuentas genera una **clase específica** de vulnerabilidades;
conocerlas es condición necesaria para auditarlas.

Cada herramienta —estático, fuzzing, simbólico— ataca una **dimensión** distinta.
Ninguna las encuentra todas sola.

Objetivos de aprendizaje

Al terminar, vas a poder:

- Nombrar y describir las vulnerabilidades del modelo de cuentas EVM
- Aplicar los patrones de defensa (CEI, reentrancy guard, pull-over-push)
- Instalar y correr **Slither** sobre un contrato propio
- Escribir un **test de invariante** en Foundry con handler
- Ubicar cada herramienta en su **capa** —estático, fuzzing, formal (el **SMTChecker** de `solc`), asistido por **LLM**, revisión humana— y saber qué aporta y qué límite tiene
- Entender qué **cubren** (y qué no) las herramientas automáticas

SEGMENTO 1 · ~40 min

Modelo de amenazas EVM

Cada bug es consecuencia del modelo

El eje conductor: estado **mutable compartido**, llamadas externas que pueden **re-entrar**, y un orden de tx decidido por **terceros**.

estado mutable

→ reentrancy, carreras

llamadas externas

→ reentrancy, delegatecall

orden de 3ros

→ front-running, MEV

El catálogo actual: OWASP SC Top 10 (2026)

Construido sobre **122 incidentes de 2025** (~US\$905 M en pérdidas):

#1 Control de acceso

el más fácil de explotar

#2 Lógica de negocio

diseño, no bug de bajo nivel

#8 Reentrancy

las herramientas ya la atrapan

Los **dos riesgos más caros** (acceso, lógica) son los que las herramientas de **patrones** peor cubren; por eso hace falta el stack completo de capas. Reentrancy se estudia igual, y primero: es el ejemplo canónico de cómo el modelo genera el bug.

Reentrancy

El contrato hace una **llamada externa antes** de actualizar su estado. Quien recibe esa llamada es código: puede volver a llamar al contrato, que todavía tiene el estado viejo.

```
// VULNERABLE: interaction antes de effect
function withdraw() external {
    uint256 amount = balances[msg.sender];
    if (amount == 0) revert InsufficientBalance();
    (bool ok,) = msg.sender.call{value: amount}(""); // ← interaction: ejecuta código ajeno
    if (!ok) revert TransferFailed();
    balances[msg.sender] = 0;                        // ← effect, demasiado tarde
}
```

Si `msg.sender` es un contrato, su `receive()` corre **dentro** del `call`, con `balances[msg.sender]` todavía intacto — y puede llamar `withdraw()` de nuevo.

Fix: Checks → Effects → Interactions.

El caso: The DAO, 17 de junio de 2016

Un fondo de inversión en un contrato: ~12 M ETH juntados (unos US\$150 M de la época).

Quien no estaba de acuerdo con una propuesta podía **separarse** con su parte, vía `splitDAO` :

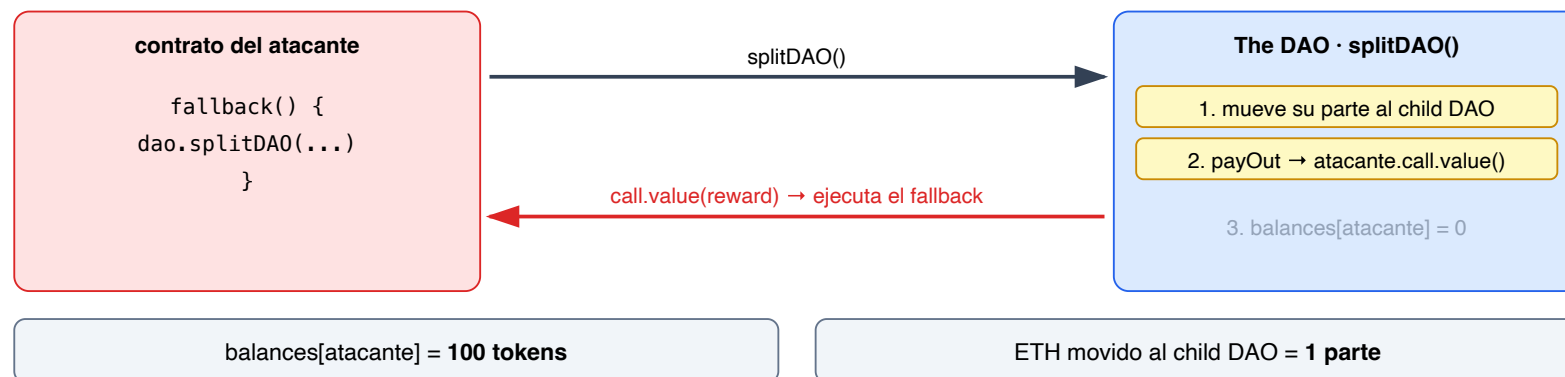
```
// DAO.sol (2016), splitDAO() – simplificado; el original tiene ~40 líneas más de chequeos
uint fundsToBeMoved = (balances[msg.sender] * splitBalance) / totalSupply;
newDAO.createTokenProxy.value(fundsToBeMoved)(msg.sender); // 1. mueve su parte al child DAO

withdrawRewardFor(msg.sender); // 2. → payOut() → _recipient.call.value(reward)()

totalSupply -= balances[msg.sender];
balances[msg.sender] = 0; // 3. demasiado tarde
paidOut[msg.sender] = 0;
```

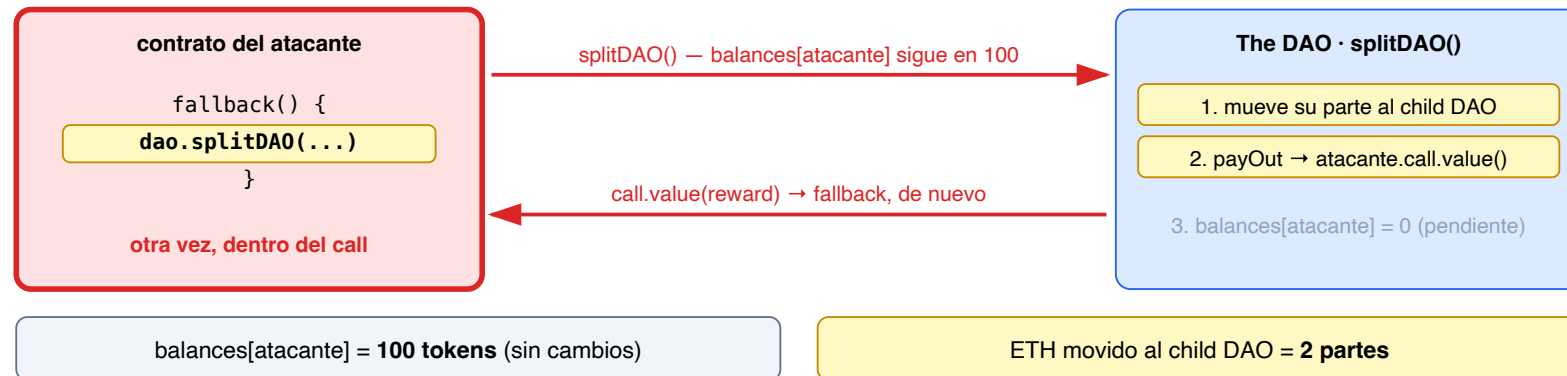
La misma forma que el `withdraw()` de la slide anterior: mover fondos (1), llamar código ajeno (2), y recién después poner el balance en cero (3).

El ataque, paso 1 — la llamada honesta



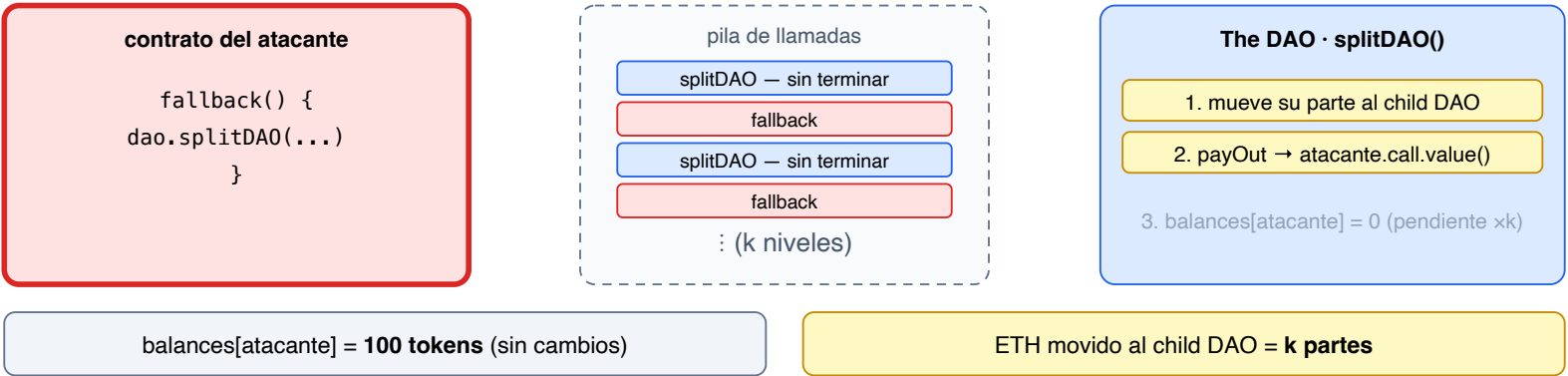
El atacante llama `splitDAO()`. El DAO calcula su parte con `balances[atacante]`, la mueve a un child DAO que él controla, y **antes de tocar `balances`** paga recompensas con un `call` al atacante. Ese `call` ejecuta el `fallback()` del atacante.

El ataque, paso 2 — la re-entrada



El `fallback()` llama `splitDAO()` **de nuevo**. El primer `splitDAO()` todavía no terminó: `balances[atacante]` sigue en 100, así que el DAO vuelve a calcular la misma parte, la vuelve a mover, y vuelve a llamar al `fallback()`.

El ataque, paso 3 — k veces



Cada nivel repite lo mismo. El límite es la profundidad de la pila de la EVM; por eso el atacante cortó cada transacción en unas decenas de re-entradas y la repitió durante horas.

El ataque, paso 4 — la pila se desenrolla



Cuando el `fallback()` deja de re-entrar, cada `splitDAO()` pendiente ejecuta su paso 3. El primero ya deja el balance en 0; los demás no cambian nada. **El ETH ya se movió k veces.**

El atacante no rompió ninguna firma ni robó ninguna clave: usó su propia cuenta y sus propios tokens. Lo único que explotó es el **orden** de tres líneas.

Lo que pasó después

- **Drenado:** ~3,6 M ETH, un tercio del fondo (~US\$50 M de la época).
- **Por qué no se perdió de inmediato:** el ETH quedó en un child DAO con un período de espera de 27 días. Ese plazo fue el tiempo para reaccionar.
- **La respuesta:** **hard fork** el 20 de julio de 2016, bloque 1.920.000. El ETH volvió a sus dueños.
- **La disidencia:** quienes rechazaron reescribir la historia siguieron en la cadena original, que hoy es **Ethereum Classic**.

Tres líneas en el orden equivocado partieron la cadena en dos. El fix era mover una línea: `balances[msg.sender] = 0` antes de la llamada externa.

Control de acceso

Una función que cambia estado privilegiado sin chequear quién la llama. Dos formas:

```
// 1. Falta el chequeo. La función que retira sí verifica al owner...
function setOwner(address nuevo) external {
    owner = nuevo;                // ...pero cualquiera puede cambiar quién es el owner
}

// 2. Se chequea contra tx.origin, que un contrato intermediario "presta"
function setOwner(address nuevo) external {
    require(tx.origin == owner);  // MAL: quien firmó la tx, no quien me llama
    owner = nuevo;
}

// BIEN: msg.sender es el llamante directo, y el chequeo está en TODAS las privilegiadas
function setOwner(address nuevo) external onlyOwner { owner = nuevo; }
```

Control de acceso: cómo se detecta

"¿Quién *debería* poder llamar a esto?" no está escrito en el código. Ningún detector lo marca como hallazgo: es el **#1 del OWASP 2026** justamente por eso.

Lo que sí hace Slither es **ponerlo a la vista**. Sobre las funciones de la slide anterior:

```
slither Acceso.sol --print vars-and-auth
```

Function	State variables written	Conditions on msg.sender
setOwner	['owner']	[]
setOwnerOrigin	['owner']	[]
setOwnerOk	['owner']	['require(bool)(msg.sender == owner)']
withdraw	['balances']	['require(bool)(balances[msg.sender] >= amount)']

Dos filas escriben `owner` con la tercera columna **vacía**: ahí está la pregunta. Y la segunda **sí tiene** un `require` — pero sobre `tx.origin`, que el printer no cuenta como condición sobre `msg.sender`. La herramienta arma la tabla; la respuesta la da quien conoce la intención.

Overflow en **unchecked** y casts

Desde 0.8 el overflow **revierte por defecto**. Pero:

```
// unchecked: el chequeo se omite explícitamente
function ajustar(uint256 amount) external {
    unchecked {
        balances[msg.sender] += amount;    // puede wraparound
    }
}

// cast: trunca en silencio
uint8 compacto = uint8(valor256);    // si valor256 > 255, resultado incorrecto
```

La vieja vulnerabilidad **no desapareció**: cambió de lugar.

delegatecall y proxies

`delegatecall` ejecuta código de **otro contrato** en el **storage del llamante**.

- Base de los proxies upgradeable (OpenZeppelin)
- También fuente de bugs sutiles: si la implementación tiene una variable en el slot 0, puede **pisar** el admin del proxy
- **Parity Wallet (2017): 513.774 ETH congelados para siempre** en 587 billeteras — no robados: **inaccesibles**
- Sigue vigente: el OWASP 2026 agregó **Proxy & Upgradeability** como categoría nueva (#10), por los proxies desplegados sin inicializar que se explotaron en serie en 2025

Slither **lo detecta** (`controlled-delegatecall` , `unprotected-upgrade`): es de los pocos bugs de esta lista donde el patrón es sintáctico. No se profundiza en el taller.

Front-running: el ejemplo mínimo

Un contrato paga 1000 ETH a quien encuentre la preimagen de un hash (*Mastering Ethereum*, cap. 9):

```
contract FindThisHash {
    bytes32 constant public hash =
        0xb5b5b97fafd9855eec9b41f74dfb6c38f5951141f9a3ecd7f44d5479b630ee0a;
    constructor() payable {} // se carga con ether
    function solve(string memory solution) public {
        require(hash == keccak256(abi.encodePacked(solution)));
        payable(msg.sender).transfer(1000 ether); // el primero que la trae, cobra
    }
}
```

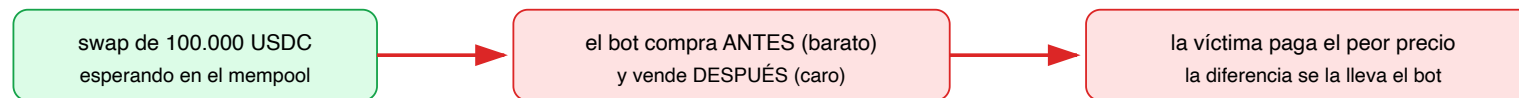
Encontrás la solución y mandás `solve("...")`. La transacción espera en el **mempool público**, con la solución **en texto claro**. Un bot la lee, manda la misma llamada con más fee, entra **antes**, y cobra. La tuya llega a un contrato vacío y revierte: pagás el gas, nada más.

El contrato no tiene ningún error, y **ninguna herramienta lo marca**: no hay patrón que buscar. El problema es del modelo: la transacción **es** la información, y el orden lo decide otro. Se mitiga, no se elimina.

Front-running / MEV

MEV = Maximal Extractable Value: lo que quien **arma el bloque** extrae reordenando, insertando o censurando tx. Consecuencia del **mempool público + orden decidido por terceros**.

Swap = cambiar un token por otro en un DEX. No hay libro de órdenes: el precio sale de la **proporción de reservas del pool**, así que **toda compra grande empuja el precio hacia arriba**.



El bot no adivina el precio: **lo mueve él**. Hay MEV **sano** (arbitraje, liquidaciones) y **tóxico** (sandwich); el sandwich es el caso a evitar.

Front-running: lo que el contrato sí puede hacer

Acotar el daño con dos parámetros que el usuario fija **antes** de que la tx sea pública:

```
function swap(uint256 amountIn, uint256 minAmountOut, uint256 deadline) external {
    if (block.timestamp > deadline) revert Expired();           // no ejecutar tarde, a otro precio
    uint256 out = _quote(amountIn);
    if (out < minAmountOut) revert Slippage(out);                // si me movieron el precio, revertir
    ...
}
```

Para no revelar la intención, **commit-reveal**: primero `commit(keccak256(solución, secreto))`, y en un bloque posterior `reveal(solución, secreto)`. Quien mira el mempool ve un hash. Es el arreglo de `FindThisHash`: el bot que copia el `reveal` ya llega tarde, el compromiso es tuyo.

Lo que NO alcanza, aunque suene razonable: *numerar* las transacciones — Ethereum ya lo hace, cada cuenta tiene su `nonce`, pero ordena **tus** tx entre sí, no contra las de un tercero: el bot corre con su cuenta y su nonce. Un *lock* tampoco: el bot lo toma primero.

Manipulación de precio: cuando el oráculo es un pool

Un protocolo lee el precio de un pool on-chain. Ese precio es **estado mutable**: con capital se mueve, y con un *flash loan* el capital sale prestado dentro de la misma transacción.

Pool (el "oráculo"): 1.000 ETH / 2.000.000 USDC → 2.000 USDC por ETH
El atacante pone 10 ETH propios. Todo lo que sigue ocurre en UNA transacción:

1. Flash loan: pide 2.000.000 USDC (sin colateral; se devuelven al final)
2. GASTA esos 2.000.000 comprando ETH en el pool y recibe 500 ETH.
¿Por qué 500 y no 1.000? Cada ETH sale más caro que el anterior:
pagó 4.000 en promedio. Pool: 500 ETH / 4.000.000 USDC → precio 8.000 (4x)
3. Deposita sus 10 ETH como colateral. El protocolo lee getReserves() y los valúa $10 \times 8.000 = 80.000$ (valen 20.000) → le presta 80.000 USDC
4. Vende los 500 ETH al pool → recupera los 2.000.000; el precio vuelve a 2.000
5. Paga el flash loan. Y NUNCA repaga los 80.000: abandona el colateral.

se llevó 80.000 · dejó 10 ETH que valen 20.000 → ganancia 60.000 USDC

Harvest Finance (oct. 2020, ~US\$34 M): los vaults valuaban leyendo un pool de Curve. El oráculo **era** el pool.

Manipulación de precio: qué lo mitiga, y hasta dónde

Mitigación	Qué anula	Qué no anula
TWAP — promedio del precio a lo largo de varios bloques	el flash loan: vive en una tx, y el promedio no se mueve en una	una manipulación sostenida varios bloques: cuesta capital real, pero es posible
Oráculo agregado (Chainlink) — muchas fuentes, muchos nodos, mediana	la manipulación del pool : no lo lee	la manipulación del mercado : reporta el precio tal cual es
Límites de sanidad — desviación máxima, topes por activo, colateral líquido	que un precio absurdo decida algo	nada por sí solos: acotan

Mango Markets (oct. 2022, ~US\$114 M): los oráculos dijeron **la verdad** sobre un mercado ilíquido que el atacante había inflado 10x con compras reales en tres exchanges. El oráculo no falló; falló aceptar como colateral algo que se movía barato.

Ninguna reemplaza a la última: **el precio que decide algo tiene que ser caro de mover.**

DoS por gas

Iterar sobre listas de longitud no acotada puede inutilizar el contrato:

```
// Si payees crece sin límite, esto eventualmente excede el gas del bloque
function distribuir() external {
    for (uint i = 0; i < payees.length; i++) {
        payees[i].transfer(shares[i]);
    }
}
```

Solución: **pull-over-push** (los usuarios retiran individualmente).

Puentes (bridges): donde está la plata

Un puente bloquea activos en una cadena y emite una representación en otra. **No es un modelo de ejecución nuevo**: es un contrato con mucho saldo y una autoridad que decide cuándo liberar.

Caso	Pérdida	Raíz
Ronin (mar. 2022)	~US\$624 M	claves del multisig comprometidas (5 de 9 firmas)
Poly Network (ago. 2021)	~US\$611 M	control de acceso: se llegó a la función que cambia los <i>keepers</i>
Wormhole (feb. 2022)	~US\$326 M	verificación de firmas incompleta → acuñar sin depositar
Nomad (ago. 2022)	~US\$190 M	una inicialización dejó válido cualquier mensaje

Ninguna es una vulnerabilidad nueva: son **control de acceso y validación de entrada** (#1 del OWASP) en el punto de mayor concentración de fondos. Y el corolario para este curso: **el modelo de ejecución no te salva** — del otro lado del puente hay un multisig, sea cuentas o EUTXO.

SEGMENTO 2 · ~20 min

Patrones de defensa

Los tres que se escriben como código

las defensas de front-running y oráculos son decisiones de diseño,
y quedaron con su vulnerabilidad

Checks-Effects-Interactions

1. CHECKS: valida condiciones (require, if/revert)
2. EFFECTS: muta el estado del contrato
3. INTERACTIONS: llama a código externo (call, transfer, otro contrato)

El orden no es estilo: es lo que cierra la reentrancy.

Reentrancy guard

```
uint256 private _status = 1;    // 1 = no reentrando

modifier nonReentrant() {
    require(_status == 1, "ReentrancyGuard: reentrant call");
    _status = 2;
    _;
    _status = 1;
}
```

OpenZeppelin lo provee listo. Costo en gas mínimo (dos SSTOREs).

Junto con CEI → **defensa en profundidad.**

Pull-over-push

```
// PUSH (peligroso con listas grandes o destinatarios que revierten):  
for (uint i = 0; i < payees.length; i++) payees[i].transfer(amounts[i]);  
  
// PULL (más seguro):  
mapping(address => uint256) public pendingWithdrawals;  
  
function withdraw() external {  
    uint256 amount = pendingWithdrawals[msg.sender];  
    pendingWithdrawals[msg.sender] = 0;  
    payable(msg.sender).transfer(amount);  
}
```

En vez de **empujar** pagos, los beneficiarios los **retiran**.

SEGMENTO 3 · ~35 min

Análisis estático

Slither y Aderyn

Qué es el análisis estático

Examina el código (o bytecode) **sin ejecutarlo**. Detecta patrones conocidos, rápido y escalable.

No puede encontrar bugs que requieren razonar sobre valores concretos de estado.
Para eso está el fuzzing.

Cómo razona un detector: seguir el flujo (taint)

No es un `grep`. Compila a un AST, construye el grafo de flujo y **sigue de dónde viene cada valor**. Un valor que controla el llamante está *contaminado*; si llega sin chequeo a un punto peligroso (*sink*), es un hallazgo.

```
function pay(address to) external onlyAdmin {  
    (bool ok,) = to.call{value: address(this).balance}(""); // sink: envío de ETH  
    require(ok);  
}
```

- `to` es una **fuentes**: lo elige quien llama → contaminado
- llega **sin sanitizar** al `call{value:}` → `arbitrary-send-eth`
- con `require(to == tesoreria)` antes, el flujo queda acotado y el hallazgo se apaga

No evalúa `address(this).balance` ni sabe quién es `msg.sender`: razona sobre la **estructura**. Por eso es rápido, y por eso no ve valores ni intención.

Slither

El analizador estándar de la industria (Trail of Bits), +100 detectores.

```
pip3 install slither-analyzer    # requiere Python 3.10+
slither --version                # slither 0.11.x

cd ethereum/
slither src/Vault.sol
```

En el Vault **correcto** (CEI), Slither **no** reporta reentrancy: el detector busca una llamada externa seguida de una escritura al estado, y en este código no la hay.

La **imagen del curso** ya lo trae. A mano, mejor en un `venv`.

Detectores útiles de Slither

Detector	Severidad	Qué encuentra
reentrancy-eth	High	reentrancy con transferencia de ETH
tx-origin	Medium	uso de tx.origin para auth
uninitialized-state	High	variables de estado sin inicializar
arbitrary-send-eth	High	envío de ETH a dirección controlada
controlled-delegatecall	High	delegatecall con arg externo

```
slither src/Vault.sol --exclude-informational --exclude-low # solo lo importante
```

Triage

Slither no devuelve "bugs": devuelve **hallazgos** por severidad. El trabajo del auditor es el **triage**: separar los reales del ruido.

Encuentra patrones, no entiende intención. Un "High" puede ser falso positivo; un "Informational" puede esconder un problema real. La severidad es guía, no veredicto.

- **Ausencia de hallazgos $\neq$ ausencia de bugs** $\rightarrow$ falso sentido de seguridad
- Los bugs de **lógica de negocio** (el escrow que no valida el output) no son un patrón sintáctico: **ninguna herramienta estática los ve**

El mismo detector `reentrancy-eth` **apagado** en el Vault con CEI y **encendido** en una versión con el orden invertido. No cambió el detector: cambió el código.

Aderyn

Analizador alternativo, escrito en **Rust** (comunidad Cyfrin):

```
curl -L https://raw.githubusercontent.com/Cyfrin/up/main/install | bash
cyfrinup                # instala/actualiza aderyn (0.6.x) → queda en ~/.cargo/bin
source ~/.cargo/env      # o abrir otra terminal
cd ethereum/ && aderyn . # genera report.md
```

Más rápido que Slither, detector set diferente, reporte en Markdown.

Buena **segunda opinión**. Para esta clase, Slither alcanza.

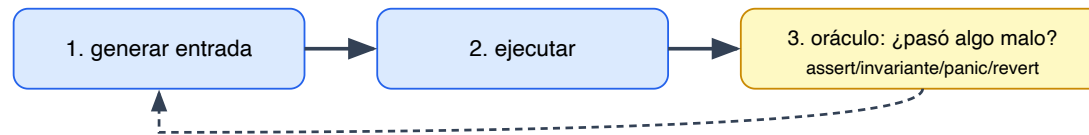
 **cargo install aderyn** da la 0.1.9 — el crate de crates.io quedó congelado seis versiones mayores atrás. El comando anda; la herramienta es de otra época.

SEGMENTO 4 · ~35 min

Fuzzing e invariantes con Foundry

Cómo funciona un fuzzer (la técnica)

Un fuzzer es un **buscador**: genera → ejecuta → consulta el **oráculo** → repite.



- **Aleatorio (Foundry por defecto)**: valores al azar con `bound` — desperdicia corridas
- **Guiado por cobertura (Echidna/Medusa)**: mide qué ramas tocó, guarda en un **corpus** lo que llegó a código nuevo y lo muta (AFL/libFuzzer). La **cobertura es la guía, no el objetivo** — no hay bug en código que nunca ejecutás

Un fuzzer NO prueba ausencia de bugs: es una **búsqueda**. "No falló en 50k corridas" es evidencia, no demostración. Para *probar* → capa simbólica/formal.

Dos modos de Foundry

Modo	Función	Qué busca
Fuzz	testFuzz_*	que un test falle para alguna entrada
Invariant	invariant_*	que una propiedad global se rompa tras una secuencia

Los fuzz tests ya los vimos (Clase 3). Hoy: **invariant testing**.

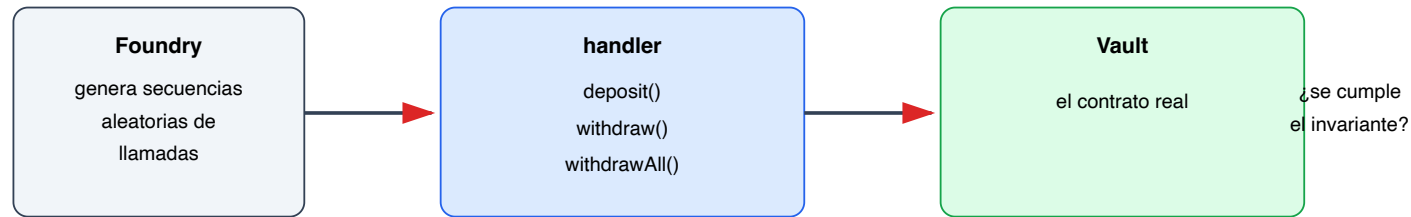
El concepto de invariante

Una propiedad que **siempre** debe ser verdadera, sin importar la secuencia de operaciones. Para el Vault:

"El ETH que el contrato custodia debe ser siempre igual a la suma de los balances internos de todos los usuarios."

Si se rompe → hay un bug (contabilidad o reentrancy).

Handler + contrato de invariantes



Archivo: `ethereum/test/VaultInvariant.t.sol`

El handler

```
contract VaultHandler is Test {
    Vault public vault;
    address[] public actors;
    mapping(address => uint256) public depositedBy;
    uint256 public totalTracked;

    function deposit(uint256 actorSeed, uint256 amount) external {
        address actor = actors[actorSeed % actors.length];
        amount = bound(amount, 1, actor.balance);
        vm.prank(actor);
        vault.deposit{value: amount}();
        depositedBy[actor] += amount;
        totalTracked += amount;          // lo que el handler cree que hay en el Vault
    }

    function withdraw(uint256 actorSeed, uint256 amount) external {
        address actor = actors[actorSeed % actors.length];
        amount = bound(amount, 1, vault.balanceOf(actor));
        vm.prank(actor);
        vault.withdraw(amount);
        depositedBy[actor] -= amount;
        totalTracked -= amount;
    }
}
```

`bound(value, min, max)` normaliza la entrada aleatoria a un rango válido. `totalTracked` es la contabilidad **paralela** del handler: suma lo que entró, resta lo que salió.

Los invariantes

```
contract VaultInvariantTest is Test {
    function setUp() public {
        vault = new Vault();
        handler = new VaultHandler(vault);
        targetContract(address(handler)); // Foundry llama al handler
    }

    function invariant_solveny() public view {
        // lo que el Vault tiene de verdad == lo que el handler contó (su `public` genera el getter)
        assertEq(address(vault).balance, handler.totalTracked());
    }
}
```

```
forge test --match-contract VaultInvariantTest -vv
# [PASS] invariant_solveny() (runs: 256, calls: 12800, reverts: ~100)
```

Si un invariante falla, Foundry no devuelve las 12.800 llamadas: **reduce** la secuencia a las pocas que hacen falta para romperlo (*shrinking*) y esa secuencia es el guion del ataque.

Echidna y Medusa: fuzzing guiado por cobertura

Fuzzers property-based **independientes** de Foundry, ambos de **Trail of Bits** (Medusa: más nuevo, en Go, **paraleliza** las campañas).

La diferencia clave con Foundry no es velocidad, es **cómo eligen las entradas**:

- Foundry: valores mayormente **aleatorios** (con `bound`)
- Echidna/Medusa: **guiados por cobertura** — miden qué ramas tocó cada entrada y **evolucionan un corpus** (guardan lo que llegó a código nuevo y lo mutan). Como AFL/libFuzzer
- → alcanzan **estados profundos** que el aleatorio rara vez arma; Echidna además integra Slither

Encuentran bugs que Foundry no alcanza con 256 corridas. No se profundizan hoy; el taller usa Foundry.

SEGMENTO 5 · ~20 min

Simbólico, formal, LLM

La idea base: ejecutar con símbolos, no valores

Corré con `amount =  $\alpha$` (cualquiera) y acumulá la **condición de camino** (PC).

En cada `if` bifurcás; en la propiedad, preguntás al **solver SMT**: ¿ `PC  $\wedge$   $\neg$ propiedad` es SAT?

```
function withdraw(uint256 amount) external {  
    if (balances[msg.sender] < amount) revert(); // PC del camino "ok":  $\beta \geq \alpha$   
    balances[msg.sender] -= amount;  
}
```

- "¿el balance queda negativo?" $\rightarrow$ SMT evalúa `$\beta \geq \alpha \wedge (\beta - \alpha) < 0$` $\rightarrow$ **UNSAT** $\rightarrow$ probado para *todo* α, β
- Si fuera **SAT**, el solver devuelve el **contraejemplo** concreto (= un PoC)

El fuzzing prueba *muchos* α ; el simbólico los cubre **todos** de una. Enemigo: **explosión de caminos** (cada `if` duplica, un loop de $n \sim \times 2^n$). Todo lo de abajo es domar esa explosión.

Taxonomía: SE, DSE, BMC, no-acotado, WP

Técnica	¿Acotado?	Herramienta
Ejecución simbólica (SE) — camino por camino	por camino	Mythril (bytecode)
DSE / concolic — corre concreto + niega ramas	por camino, con valores	Manticore
BMC — desenrolla loops hasta k	acotado ($\leq k$ pasos)	SMTChecker <code>bmc</code> , halmos
Invariante inductivo (CHC/Horn) — lo infiere	no acotado	SMTChecker <code>chc</code>
WP contra spec — VC <code>Pre $\Rightarrow$ WP(Prog, Post)</code>	unroll (acot.) o invariante (no)	Certora (CVL)

- **WP** (precondición más débil): calcula *hacia atrás* qué debe valer; el `while` **necesita invariante** de loop (sin él, se desenrolla $\rightarrow$ vuelve acotado)
- Certora genera las VC desde tu spec CVL y las descarga con SMT — estándar en DeFi (Aave, Maker)

Límite común: **explota en tiempo** y solo prueba lo que supiste especificar. Por eso va **con fuzzing**, no en vez de. El SMTChecker (siguiente) expone los dos motores: `bmc` y `chc`.

El verificador YA viene en el compilador: SMTChecker

Verificación formal **dentro de solc**, en tiempo de compilación (mismo enfoque simbólico, pero integrado en el build que ya corrés).

```
# foundry.toml - corre con `forge build`
[profile.default.model_checker]
contracts      = { "src/Vault.sol" = ["Vault"] }
engine         = "chc"           # chc (no acotado) / bmc
targets        = ["assert", "overflow", "underflow", "divByZero"]
show_unsupported = true          # ← si no, no dice QUÉ no soporta
show_unproved   = true
show_proved_safe = true
```

El bloque **ya está en ethereum/foundry.toml**, comentado — suma ~20 s a cada build.

Para la demo: **descomentalo** y `forge build --force` (`forge` cachea: sin `--force` no recompila).

⚠ Depende del **build de solc**, no de la arquitectura en sí: en **amd64** anda (la imagen del curso trae la `libz3` que solc carga); en **arm64** el binario de solc viene **sin z3** y hace falta **Eldarica** como solver externo (`solvers = ["eld"]`).

- **Automático:** overflow/underflow, div-by-zero, `assert`; modela llamadas externas como re-entrantes
- **Tuyo:** cualquier `assert` (p. ej. solvencia) → lo **prueba** o te da un **contraejemplo**

Demo: lo corremos sobre *nuestro* Vault (el correcto)

```
Warning (8364): Assertion checker does not yet implement type error
--> src/Vault.sol:27:  revert InsufficientBalance();          (×4, una por custom error)

Warning (3944): CHC: Underflow (resulting value less than 0) happens here.
Counterexample:  amount = 1
Transaction trace: Vault.constructor() → Vault.withdraw(1)
--> src/Vault.sol:30:  balances[msg.sender] -= amount;
```

El Vault de CEI, con **23 tests en verde**, y el verificador del compilador dice que `withdraw(1)` sobre un balance vacío llega a la resta. Tarda ~20 s.

Es un falso positivo — y la causa está en la advertencia de arriba.

Por qué se equivoca: no entiende los custom errors

El SMTChecker **no modela** `revert MiError()`. En su código fuente, `revert()` se traduce a `require(false)` —el camino termina—, pero una llamada a un error custom cae en `case FunctionType::Kind::Error: // This can be safely ignored.` Se saltea.

```
if (balances[msg.sender] < amount) revert InsufficientBalance(); // ← salteado: el camino sigue
balances[msg.sender] -= amount;                               // ← "underflow"
```

Cambiando los 4 custom errors por `require(...)`, mismo contrato:

```
Info (9576): CHC: Underflow (resulting value less than 0) check is safe!
```

De *"encontré un bug"* a **demostré** que no puede pasar. Mismo código, distinto azúcar.

Verificado en `solc` 0.8.26, 0.8.29 y 0.8.30: el "not yet" sigue vigente.

Las tres lecciones de ese falso positivo

1. Las herramientas formales tienen un subconjunto soportado del lenguaje.

Fuera de él no fallan ruidosamente: **pierden precisión en silencio**.

La Clase 2 enseña custom errors como práctica moderna... y el verificador no los soporta.

2. Hay un segundo hallazgo, de otra naturaleza: en `deposit()`,

`+= msg.value` *podría* pasarse de `2**256-1`. Formalmente cierto, prácticamente imposible (no existe tanto ETH). **Válido e irrelevante**.

3. El verbo: `happens here` (contraejemplo concreto) $\neq$ `might happen`

(no pudo probarlo ni refutarlo) $\neq$ `proved safe` (lo demostró). **Son tres estados**.

"El verificador no dijo nada" y "el verificador probó que es seguro"
son cosas **muy** distintas.

LLM: de asistente a agente (2024 → 2026)

Cubren el hueco #2 del OWASP 2026: la **lógica de negocio**, que ni el estático ni el SMTChecker ven.
El campo se movió rápido — tres generaciones:

Generación	Ejemplo	Idea
1. LLM directo	Claude / GPT con consigna	primera pasada + explicar código
2. Híbrido LLM + análisis	GPTScan (2024), iAudit (2025)	LLM propone, el análisis confirma
3. Agente autónomo	AuditAgent (Nethermind, 2025-26)	recorre el repo, itera, redacta el reporte

Hasta hay híbridos formal+LLM: **Concordance** (Certora) reescribe código optimizado y **prueba** que la reescritura es equivalente.

¿Cómo validan que la propuesta es correcta?

Un LLM solo **NO valida nada** (por eso alucina). La herramienta seria **acopla la propuesta a un oráculo externo** — de menor a mayor garantía:

Estrategia	Quién valida	Ejemplo
Confirmación con análisis de programa	chequeo de flujo estático	GPTScan
PoC ejecutado	la ejecución (corre el exploit)	agentes / EVMbench
Prueba formal de equivalencia	la matemática	Concordance (Certora)

El LLM propone, un oráculo dispone. Donde no hay oráculo, no hay validación: hay una hipótesis. Cada hallazgo de un LLM se confirma o se descarta con un test en Foundry.

¿Cuán buenos? La advertencia, ahora con datos

- **EVMbench** (OpenAI + Paradigm, feb. 2026; 117 bugs de 40 auditorías): el mejor modelo **explota el 72 %** de los críticos; el de seis meses antes, 32 %; al arrancar el proyecto, menos del 20 %. **Mejora muy rápido.**
- **AuditAgent** (Nethermind, sobre auditorías reales): **30 % de recall** promedio vs auditores humanos —deja $\frac{2}{3}$ afuera—, hasta 50 % en algunos proyectos; en el 62 % encontró al menos un problema válido.

Alucina · no determinista · **no** produce PoC. Un hallazgo del LLM es una **hipótesis** → confirmar con un test. Así lo usan las firmas serias: el agente adelanta, el humano decide.
Todo número acá es una foto de 2026 — la técnica es estable, las cifras cambian.

Síntesis: las herramientas son capas, no rivales

No compiten: se **encadenan**, y cada una cubre una dimensión distinta del espacio de bugs.

Capa	Herramienta	Encuentra	Su límite
Estático	Slither, Aderyn	patrones, rápido	no razona valores ni intención
Fuzzing	Foundry, Echidna, Medusa	propiedades con valores	no exhaustivo
Simbólico / formal	SMTChecker, halmos, Certora	prueba para <i>todas</i> las entradas	explota en tiempo; hay que escribir la spec
Asistido por LLM	Claude/GPT, agentes, GPTScan	candidatos de lógica de negocio	alucina, sin PoC, ~1/3 recall
Humano	vos	intención, lógica de negocio	no escala

Cada capa tapa los huecos de la anterior. La revisión **humana** está por encima: es la única que entiende qué *debería* hacer el contrato. **Ninguna sola alcanza.**

Actividad

1. Instalar Slither y correrlo sobre `Vault.sol`. Clasificar los hallazgos: ¿informational? ¿riesgo real?
2. Correr los invariantes (`forge test --match-contract VaultInvariantTest -vv`). ¿Cuántas llamadas? ¿Cuántos reverts y por qué no son fallas?
3. **Análisis:** si un actor deposita más de lo que el handler le asignó, ¿el invariante fallaría? ¿Por qué no?
4. **Reproducir el falso positivo del SMTChecker.** Descomentar el bloque `model_checker` de `foundry.toml`, `forge build --force`. (a) ¿Por qué es un falso positivo? (b) ¿Qué advertencia lo explica? (c) Cambiar los 4 custom errors por `require(...)` : ¿qué cambia en la salida?
5. **Ubicar cada herramienta en su capa:** para Slither, el fuzzer, el SMTChecker y un LLM, una frase: qué capa cubre, un bug que *sí* encontraría y uno que *no*.

Lecturas

- **OWASP Smart Contract Top 10 (2026):** scs.owasp.org/sctop10 — el catálogo vigente
- ***Mastering Ethereum*, 2ª ed., cap. 9 "Smart Contract Security"** — cada vulnerabilidad con técnica de prevención y caso real
- Phil Daian, *Analysis of the DAO exploit* (2016) — el ataque, paso a paso, sobre el código real
- **Slither:** github.com/crytic/slither — README y wiki de detectores
- **Foundry Book** — "Invariant Testing" · **SMTChecker:** docs de Solidity

Próxima clase

Clase 7 — Taller de análisis (Ethereum)

Una versión del Vault con el orden de `withdraw()` invertido: escribir el contrato que la drena, dejar que el invariante lo encuentre solo, arreglarla y verificar el fix.

Para prepararse: releer `Vault.sol` y pensar *¿qué línea habría que mover para crear la vulnerabilidad?*